

18-799: Applied Computer Vision

Spring 2026

CNN Architecture

Contents

- History of CNN
- Evolution and Milestones in CNN Architecture
 - LeNet
 - AlexNet
 - VGG
 - GoogleNet
 - ResNet

History of CNN

- Hubel and Wiesel's research in the 1950s and 1960s demonstrated the presence of neurons in cat visual cortices that react to specific small areas in the visual field.
- When the eyes are stationary, the specific area of visual space that influences the activity of an individual neuron is referred to as its receptive field.
- Adjacent cells exhibit receptive fields that are alike and overlap with each other.
- Across the cortex, the size and position of receptive fields systematically change to create a complete map of visual space.

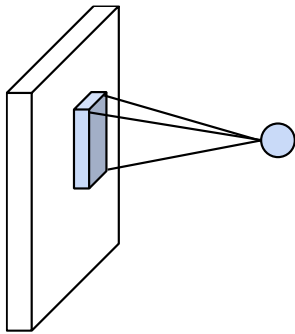
(Hubel and Wiesel, 1959)

History of CNN

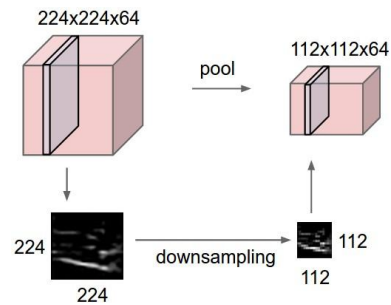
- Inspired by the work of Hubel and Wiesel, Kunihiro Fukushima introduced the "neocognitron" in 1980.
- The neocognitron introduced the two basic types of layers in CNNs:
 - A convolutional layer which contains units whose receptive fields cover a patch of the previous layer.
 - Down sampling layers which contain units whose receptive fields cover patches of previous convolutional layers.

Components of a modern CNN Architecture

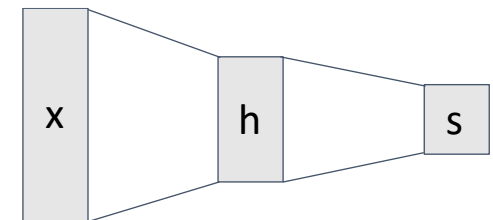
Convolution Layers



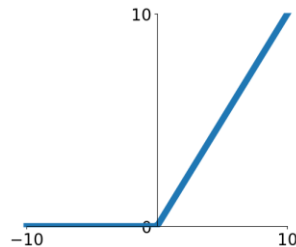
Pooling Layers



Fully-Connected Layers



Activation Function



Convolution Summary

Input: $C_{in} \times H \times W$

Hyperparameters:

- Kernel size: $K_H \times K_W$
- Number filters: C_{out}
- Padding: P
- Stride: S

Weight matrix: $C_{out} \times C_{in} \times K_H \times K_W$ giving C_{out} filters of size $C_{in} \times K_H \times K_W$

Bias vector: C_{out}

Output size: $C_{out} \times H' \times W'$ where:

$$H' = (H - K + 2P)/S + 1$$

$$W' = (W - K + 2P)/S + 1$$

Convolution Summary

- **Common settings:**

$K_H = K_W$ (Small square filters)

$P = (K - 1)/2$ ("Same" padding)

$K = 3, P = 1, S = 1$ (3×3 conv)

$K = 5, P = 2, S = 1$ (5×5 conv)

$K = 1, P = 0, S = 1$ (1×1 conv)

$K = 3, P = 1, S = 2$ (Downsample by 2)

Convolution Summary

- Input volume: $3 \times 32 \times 32$
10 5×5 filters with stride 1 , pad 2
- Output volume size:
 - $10 \times 32 \times 32$
- Number of learnable parameters:
 - 760
- Number of multiply-add operations:
 - **768,000**
 $10 * 32 * 32 = 10,240$ outputs; each output is the inner product of two $3 \times 5 \times 5$ tensors (75 elements); total = $75 * 10240 = 768\text{ K}$

Convolution Summary

- Classic architecture:
 - [Conv, ReLU, Pool] x N, flatten, [FC, ReLU] x N, FC
 - LeNet-5
 - [Conv, ReLU, Pool] x 2, flatten, FC, ReLU FC

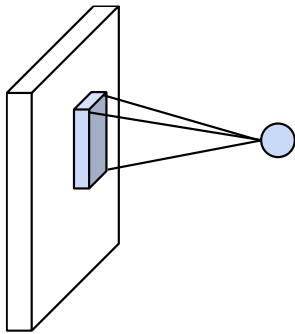
LeNet-5

Layer	Output Size	Weight Size
Input	$1 \times 28 \times 28$	
Conv ($C_{\text{out}} = 20, K = 5, P = 2, S = 1$)	$20 \times 28 \times 28$	$20 \times 1 \times 5 \times 5$
ReLU	$20 \times 28 \times 28$	
MaxPool ($K = 2, S = 2$)	$20 \times 14 \times 14$	
Conv ($C_{\text{out}} = 50, K = 5, P = 2, S = 1$)	$50 \times 14 \times 14$	$50 \times 20 \times 5 \times 5$
ReLU	$50 \times 14 \times 14$	
MaxPool ($K = 2, S = 2$)	$50 \times 7 \times 7$	
Flatten	2450	
Linear (2450 $\rightarrow$ 500)	500	2450×500
ReLU	500	
Linear (500 $\rightarrow$ 10)	10	500×10

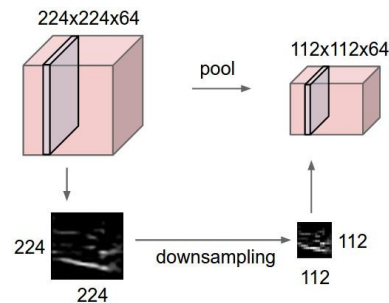
- As we go through the network:
- Spatial size decreases (using pooling or strided conv)
- Number of channels increases (total "volume" is preserved!)

Components of a modern CNN Architecture

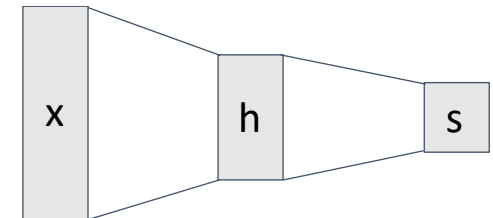
Convolution Layers



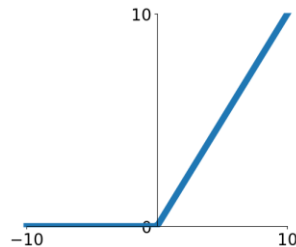
Pooling Layers



Fully-Connected Layers



Activation Function



Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

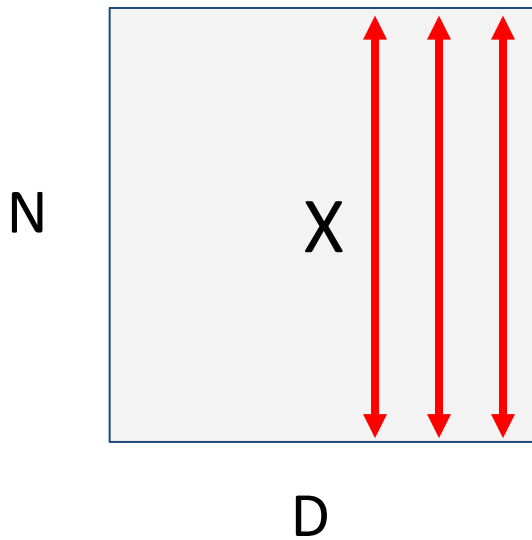
Batch Normalization

- Idea: "Normalize" the outputs of a layer so they have zero mean and unit variance
- Why? Helps reduce "internal covariate shift", improves optimization
- The following is batch normalization:

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Batch Normalization

Input: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

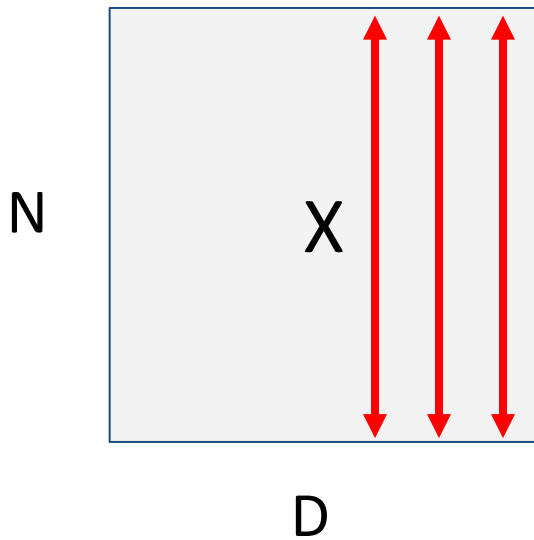
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

Batch Normalization

Input: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Problem: What if zero-mean, unit variance is too hard of a constraint?

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

Batch Normalization

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$,
 $\beta = \mu$ will recover the
identity function!

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Batch Normalization for ConvNets

Batch Normalization for
fully-connected networks

$$\mathbf{x}: \mathbf{N} \times \mathbf{D}$$

Normalize



$$\boldsymbol{\mu}, \boldsymbol{\sigma}: \mathbf{1} \times \mathbf{D}$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta}: \mathbf{1} \times \mathbf{D}$$

$$\mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

Batch Normalization for
convolutional networks

$$\mathbf{x}: \mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W}$$

Normalize

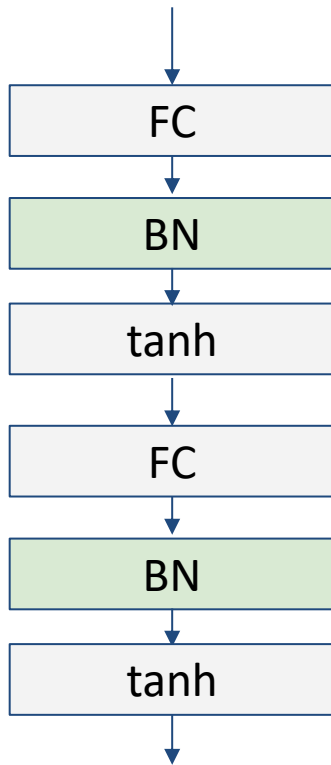


$$\boldsymbol{\mu}, \boldsymbol{\sigma}: \mathbf{1} \times \mathbf{C} \times \mathbf{1} \times \mathbf{1}$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta}: \mathbf{1} \times \mathbf{C} \times \mathbf{1} \times \mathbf{1}$$

$$\mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

Batch Normalization



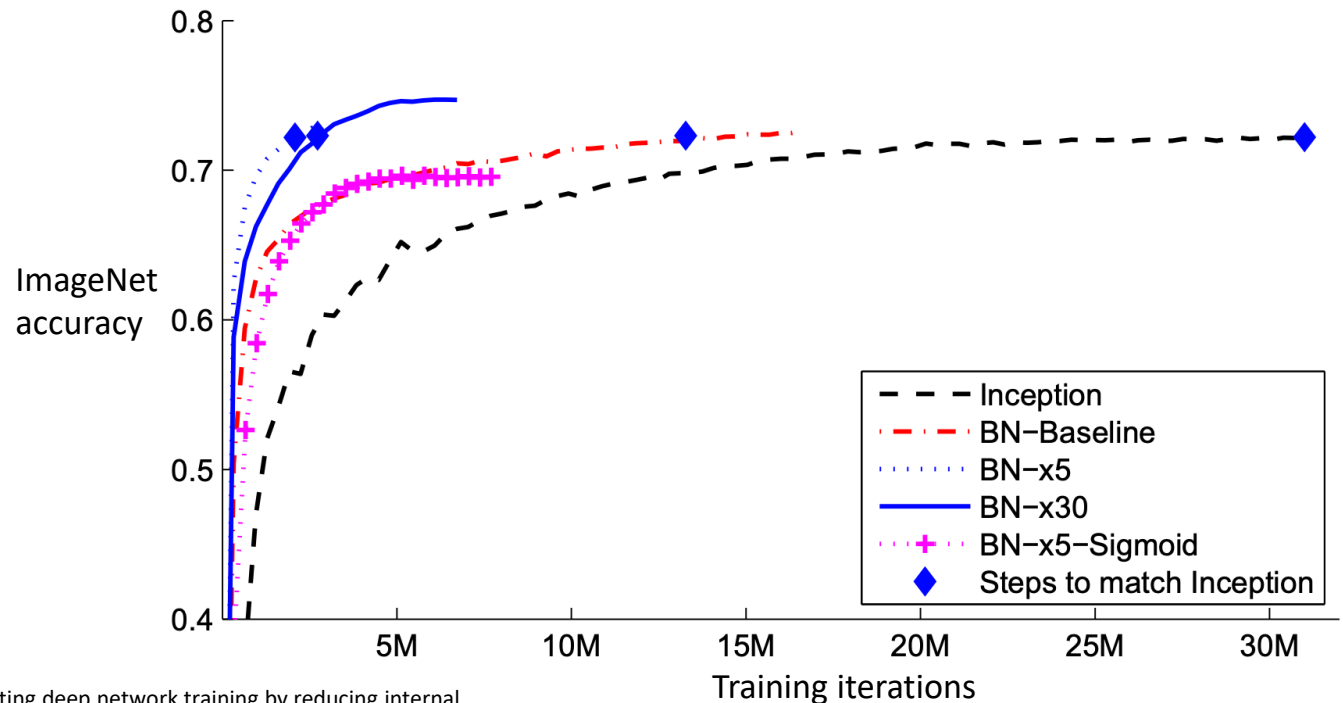
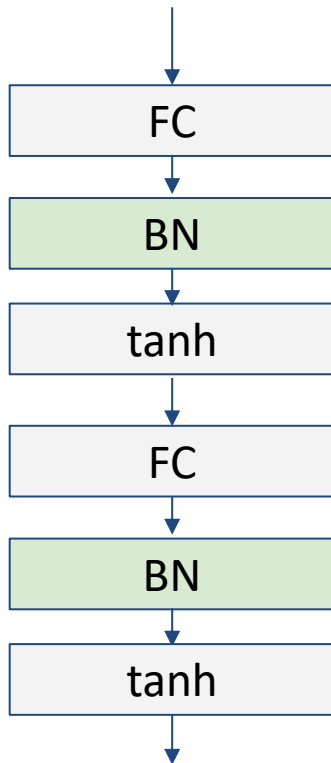
Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

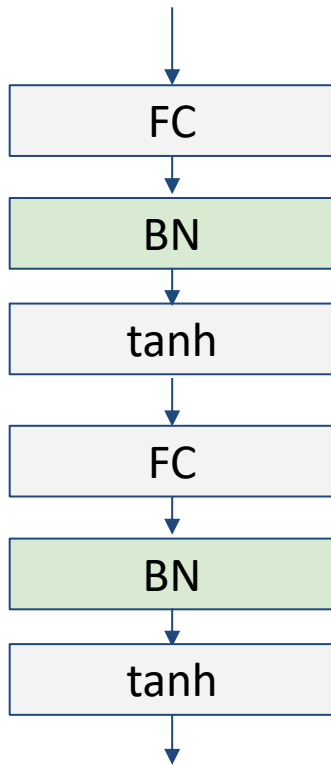
Batch Normalization

- Makes deep networks **much** easier to train!
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!



Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

Batch Normalization

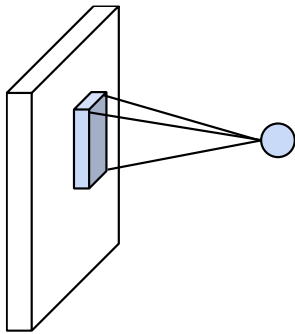


- Makes deep networks **much** easier to train!
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!
- Not well-understood theoretically (yet)
- Behaves differently during training and testing

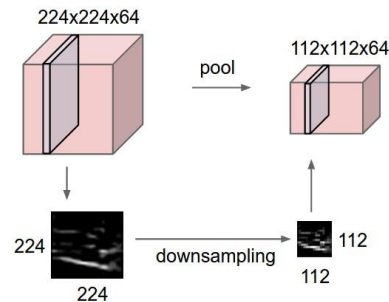
Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

Components of a Convolutional Network

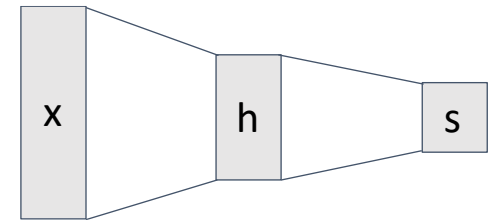
Convolution Layers



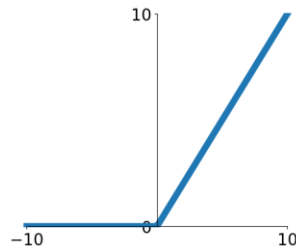
Pooling Layers



Fully-Connected Layers



Activation Function



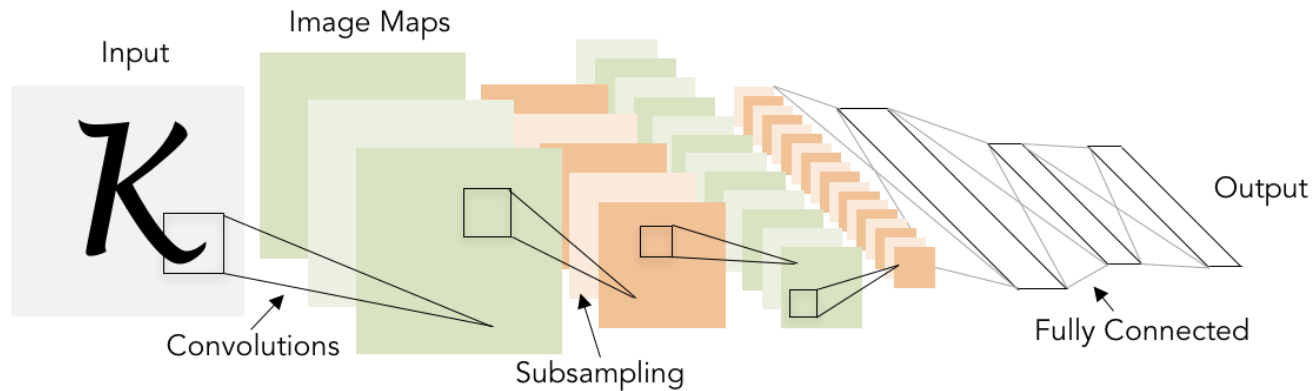
Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Convolutional Networks

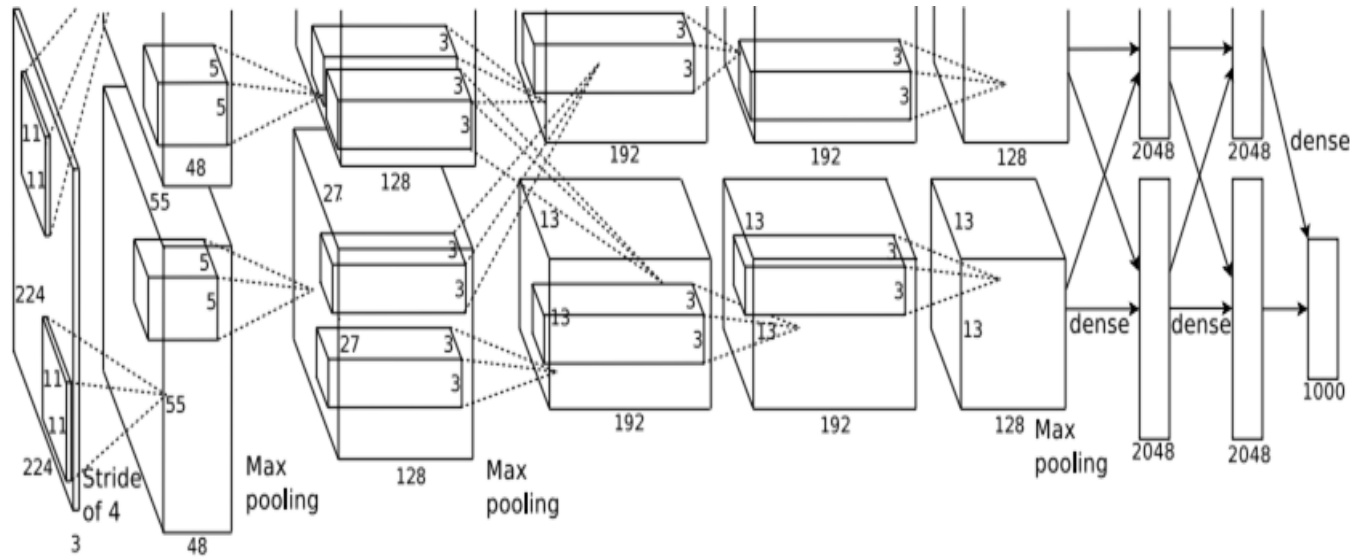
Classic architecture: [Conv, ReLU, Pool] x N, flatten, [FC, ReLU] x N, FC

LeNet-5



Lecun et al, "Gradient-based learning applied to document recognition", 1998

AlexNet



5 Convolutional layers
Max pooling
3 fully-connected layers
ReLU nonlinearities

Used “Local response normalization”;
Not used anymore

Trained on two GTX 580 GPUs – only
3GB of memory each! Model split
over two GPUs

AlexNet

	Input size		Layer				Output size	
Layer	C	H / W	filters	kernel	stride	pad	C	H / W
conv1	3	227	64	11	4	2	?	

AlexNet

	Input size		Layer				Output size	
Layer	C	H / W	filters	kernel	stride	pad	C	H / W
conv1	3	227	64	11	4	2	64	?

Recall: Output channels = number of filters

AlexNet

	Input size		Layer				Output size	
Layer	C	H / W	filters	kernel	stride	pad	C	H / W
conv1	3	227	64	11	4	2	64	56

$$\begin{aligned}\text{Recall: } W' &= (W - K + 2P) / S + 1 \\ &= (227 - 11 + 2 \cdot 2) / 4 + 1 \\ &= 220/4 + 1 = 56\end{aligned}$$

AlexNet

	Input size		Layer				Output size		
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)
conv1	3	227	64	11	4	2	64	56	?

AlexNet

	Input size		Layer				Output size		
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)
conv1	3	227	64	11	4	2	64	56	784

$$\begin{aligned}\text{Number of output elements} &= C * H' * W' \\ &= 64 * 56 * 56 = 200,704\end{aligned}$$

$$\text{Bytes per element} = 4 \text{ (for 32-bit floating point)}$$

$$\begin{aligned}\text{KB} &= (\text{number of elements}) * (\text{bytes per elem}) / 1024 \\ &= 200704 * 4 / 1024 \\ &= \mathbf{784}\end{aligned}$$

AlexNet

	Input size		Layer				Output size			
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)
conv1	3	227	64	11	4	2	64	56	784	?

AlexNet

	Input size		Layer				Output size			
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)
conv1	3	227	64	11	4	2	64	56	784	23

$$\begin{aligned}\text{Weight shape} &= C_{\text{out}} \times C_{\text{in}} \times K \times K \\ &= 64 \times 3 \times 11 \times 11\end{aligned}$$

$$\text{Bias shape} = C_{\text{out}} = 64$$

$$\begin{aligned}\text{Number of weights} &= 64 \times 3 \times 11 \times 11 + 64 \\ &= \mathbf{23,296}\end{aligned}$$

AlexNet

	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	?

AlexNet

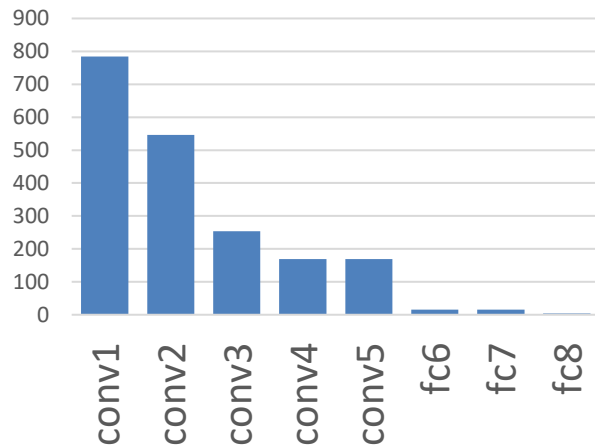
	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73

Number of floating point operations (multiply+add)
= (number of output elements) * (ops per output elem)
= $(C_{out} \times H' \times W') * (C_{in} \times K \times K)$
= $(64 * 56 * 56) * (3 * 11 * 11)$
= $200,704 * 363$
= **72,855,552**

AlexNet

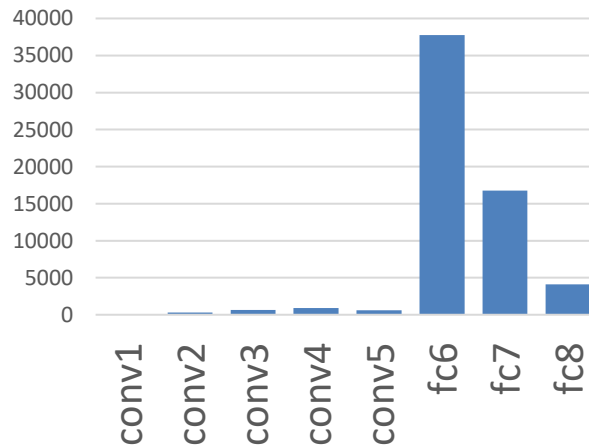
Most of the **memory usage** is in the early convolution layers

Memory (KB)



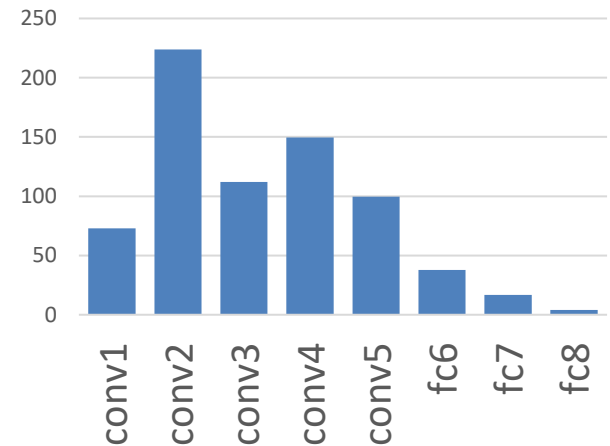
Nearly all **parameters** are in the fully-connected layers

Params (K)

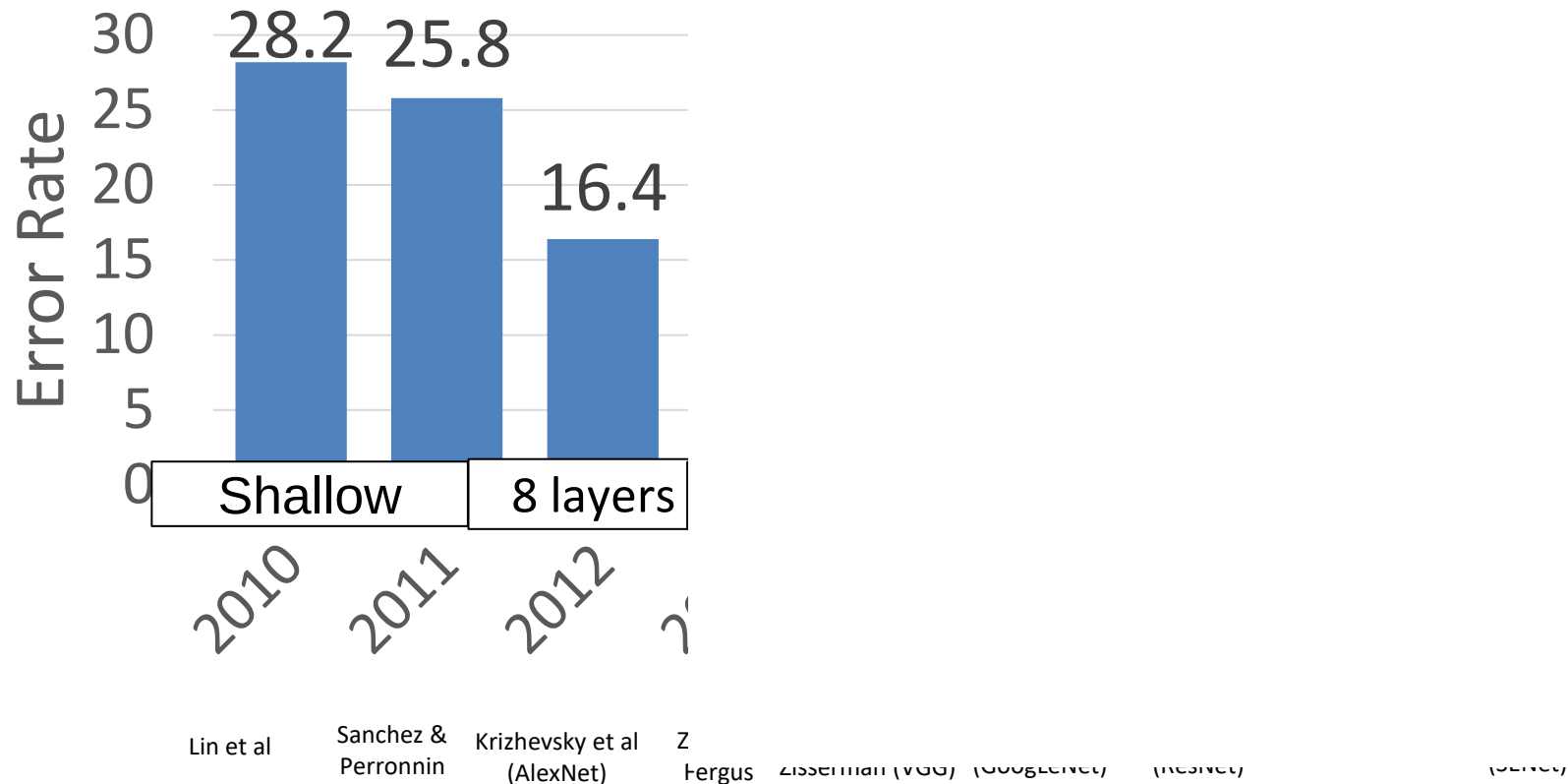


Most **floating-point ops** occur in the convolution layers

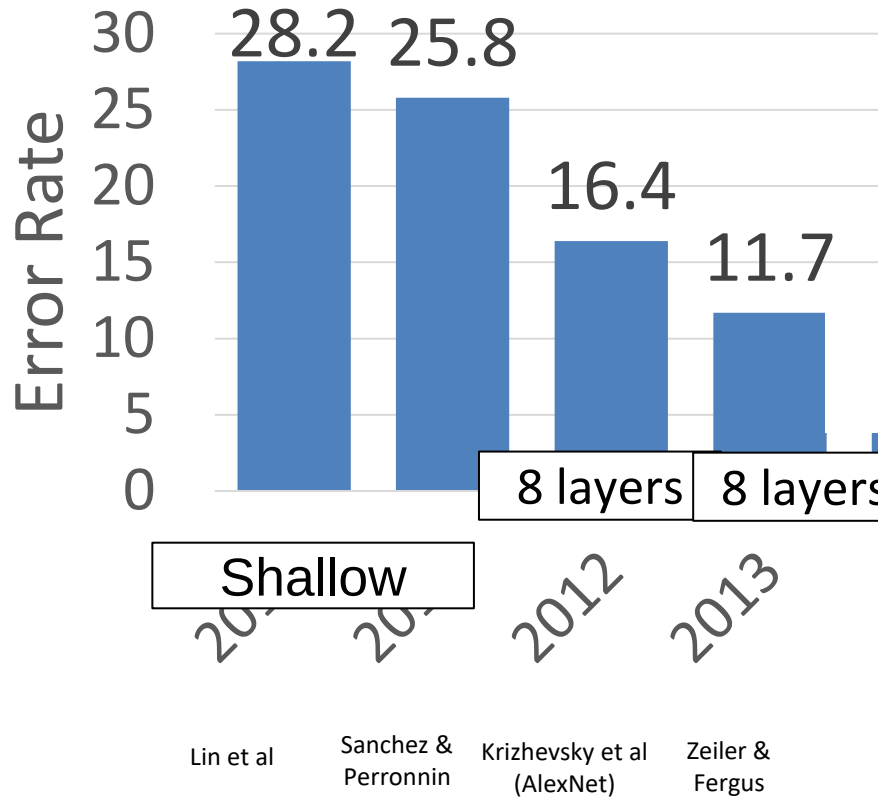
MFLOP



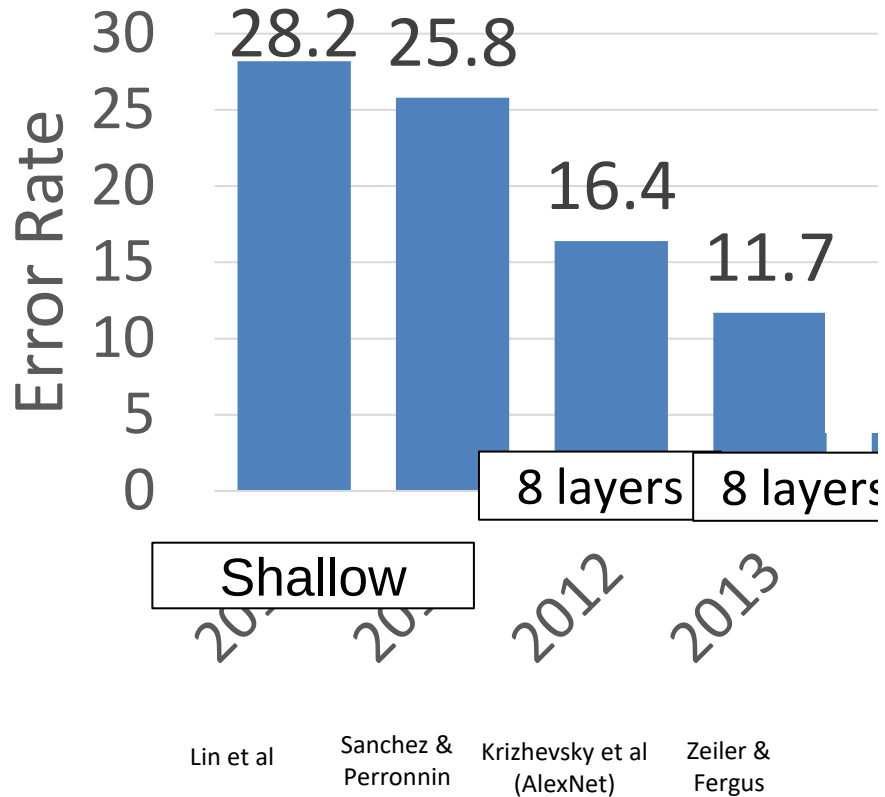
ImageNet Classification Challenge



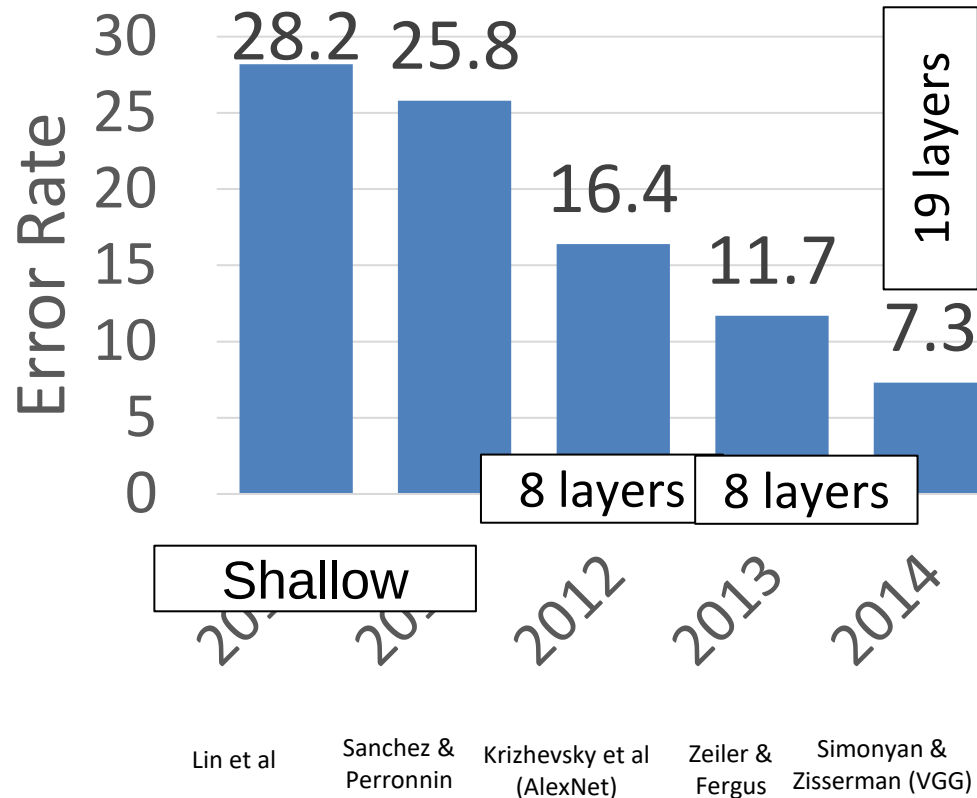
ImageNet Classification Challenge



ImageNet Classification Challenge



ImageNet Classification Challenge



VGG: Deeper Networks, Regular Design

VGG Design rules:

All conv are 3x3 stride 1 pad 1

All max pool are 2x2 stride 2

After pool, double #channels

Network has 5 convolutional **stages**:

Stage 1: conv-conv-pool

Stage 2: conv-conv-pool

Stage 3: conv-conv-pool

Stage 4: conv-conv-conv-[conv]-pool

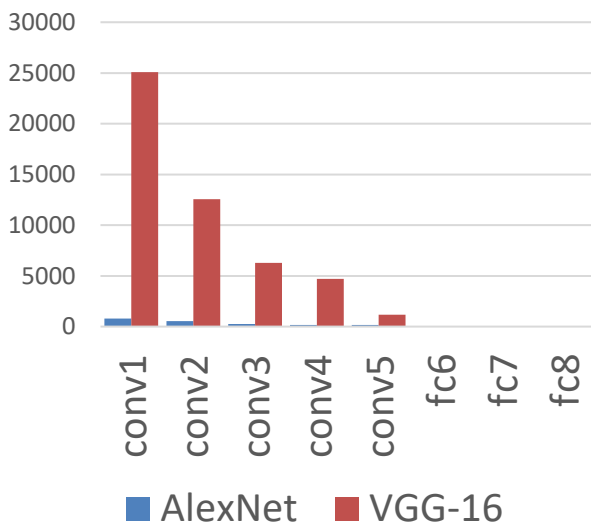
Stage 5: conv-conv-conv-[conv]-pool

(VGG-19 has 4 conv in stages 4 and 5)

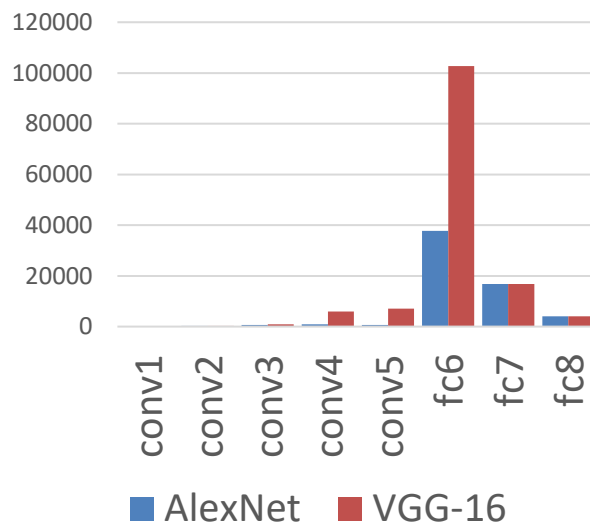


AlexNet vs VGG-16: Much bigger network!

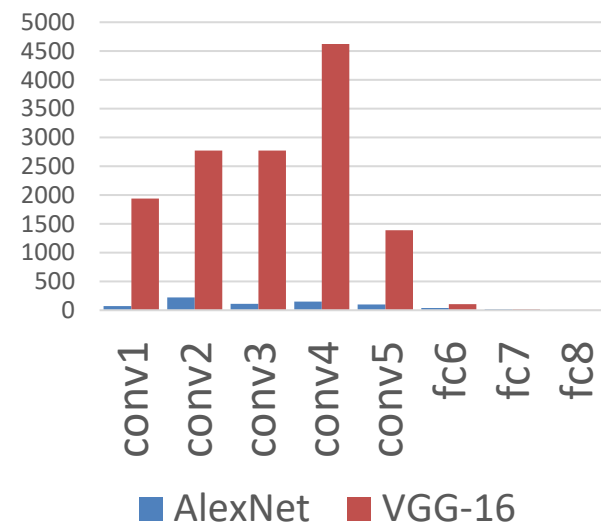
AlexNet vs VGG-16
(Memory, KB)



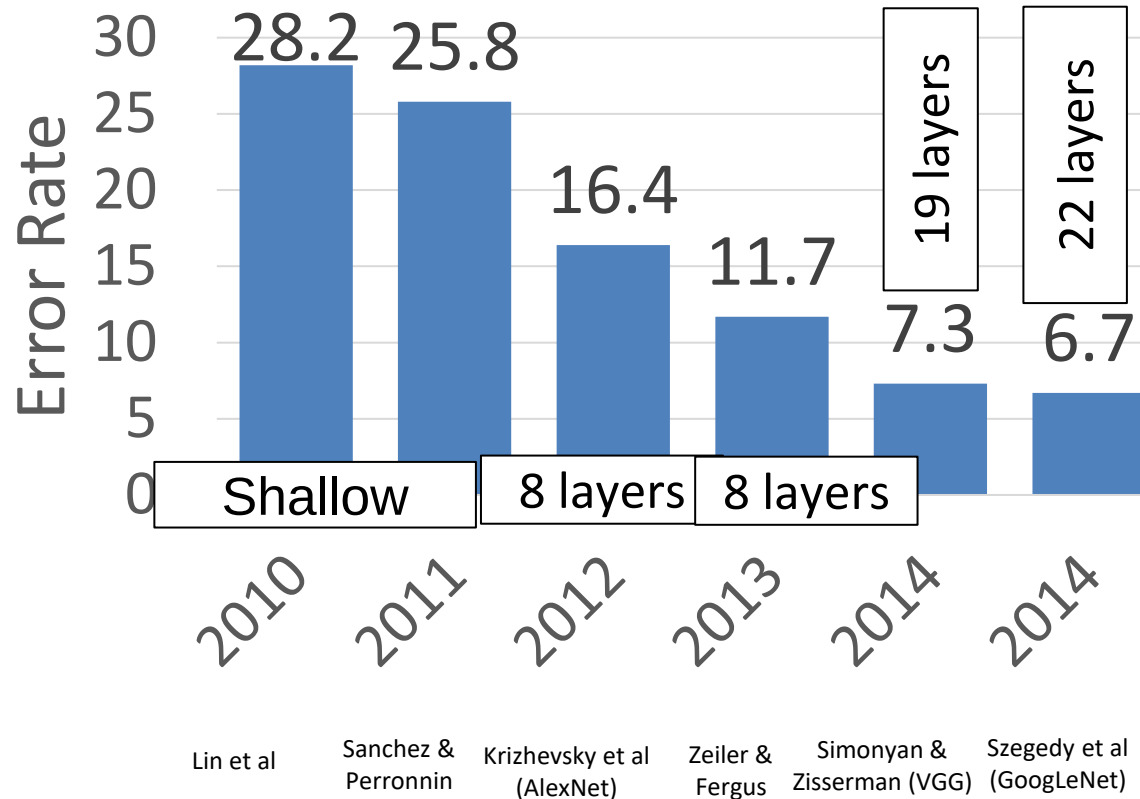
AlexNet vs VGG-16
(Params, M)



AlexNet vs VGG-16
(MFLOPs)



ImageNet Classification Challenge



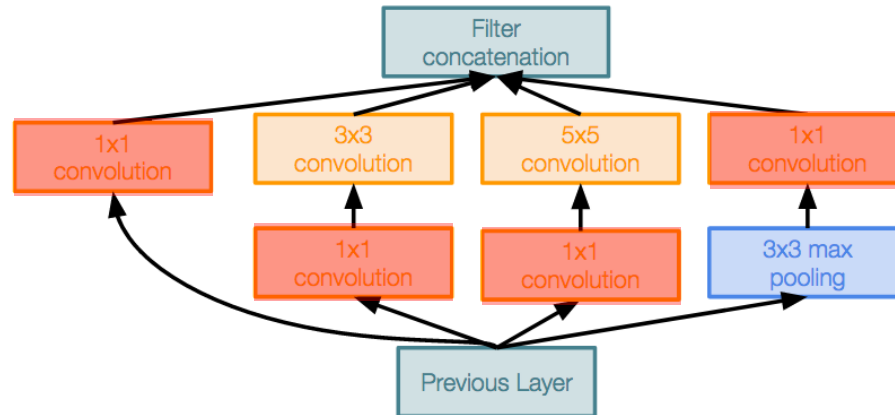
GoogLeNet: Inception Module

Inception module

Local unit with parallel branches

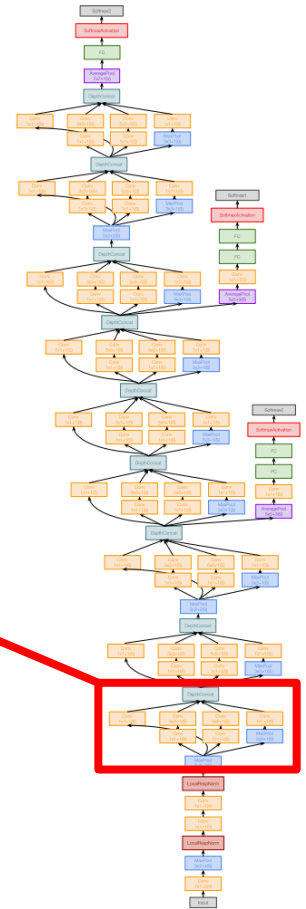
Local structure repeated many times throughout the network

Uses 1x1 “Bottleneck” layers to reduce channel dimension before expensive conv (we will revisit this with ResNet!)



Related to (sum of) separable filters

Perona, “Deformable kernels for early vision”. PAMI 1995

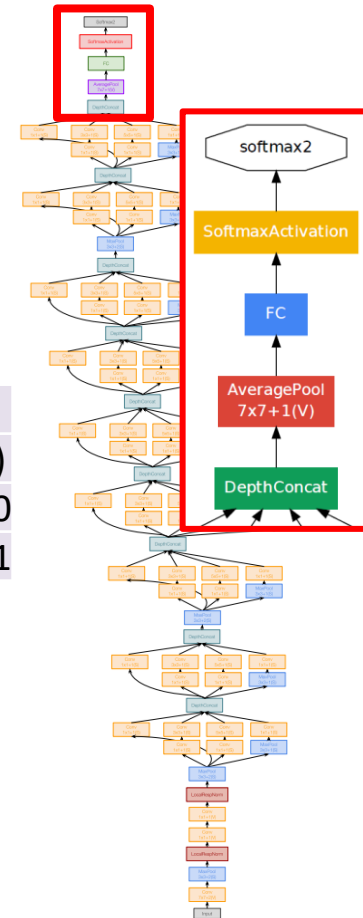


Szegedy et al, “Going deeper with convolutions”, CVPR 2015

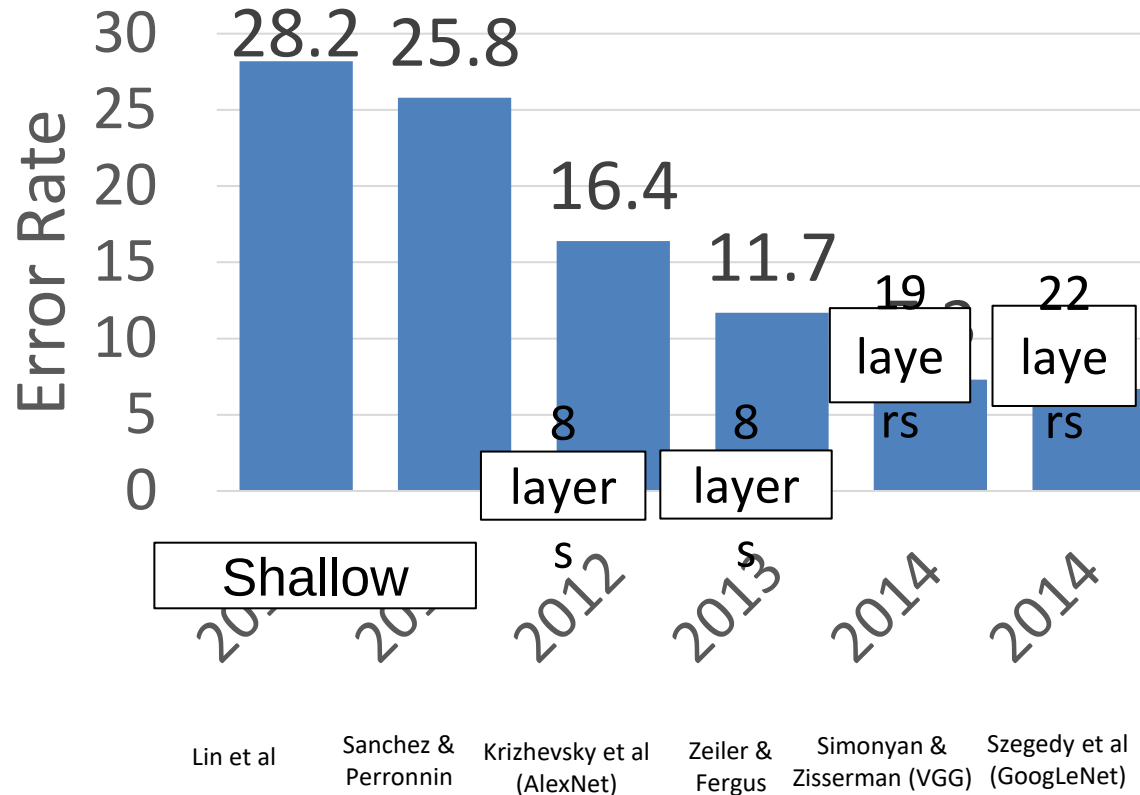
GoogLeNet: Global Average Pooling

No large FC layers at the end! Instead uses **global average pooling** to collapse spatial dimensions, and one linear layer to produce class scores
(Recall VGG-16: Most parameters were in the FC layers!)

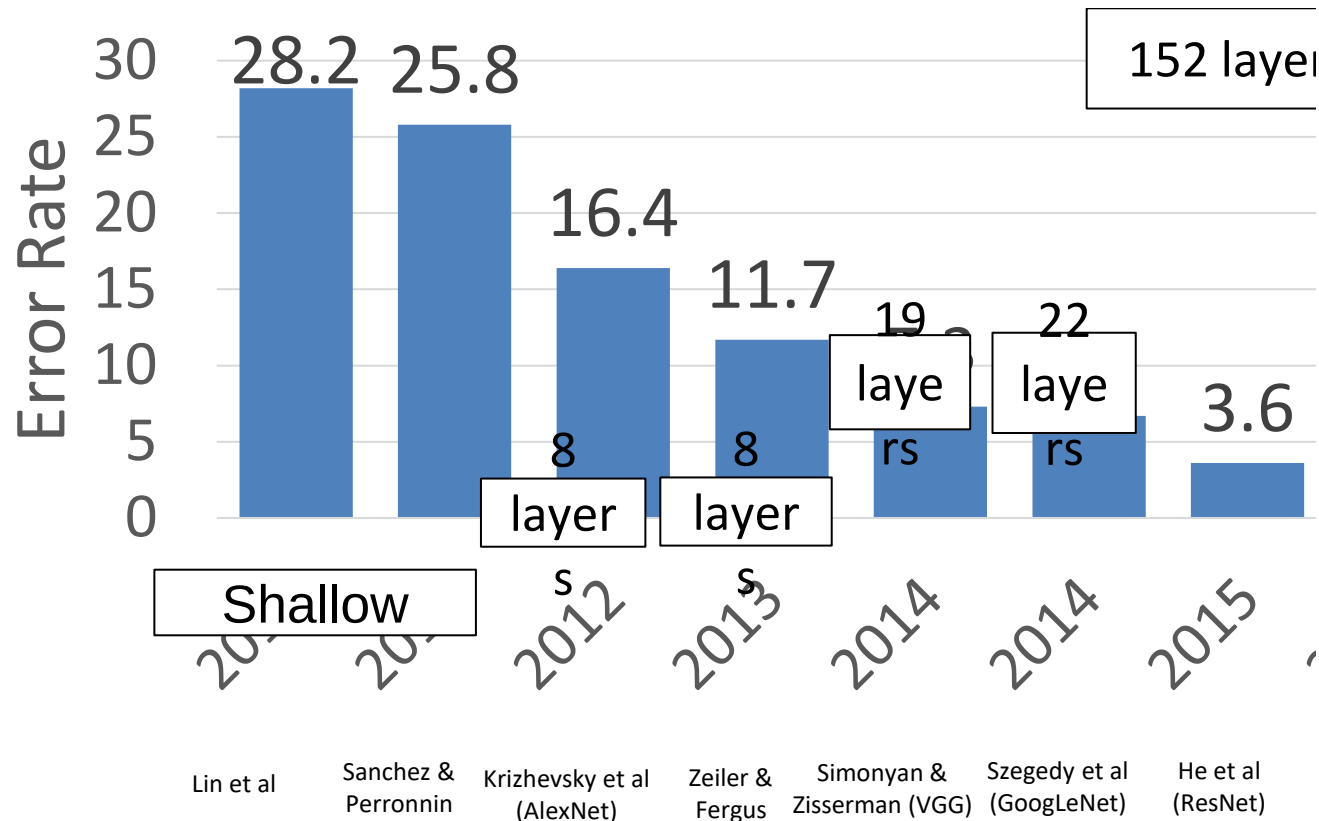
	Input size		Layer				Output size				
Layer	C	H/W	filters	kernel	stride	pad	C	H/W	memory (KB)	params (k)	flop (M)
avg-pool	1024	7		7	1	0	1024	1	4	0	0
fc	1024		1000				1000		0	1025	1



ImageNet Classification Challenge



ImageNet Classification Challenge



Residual Networks

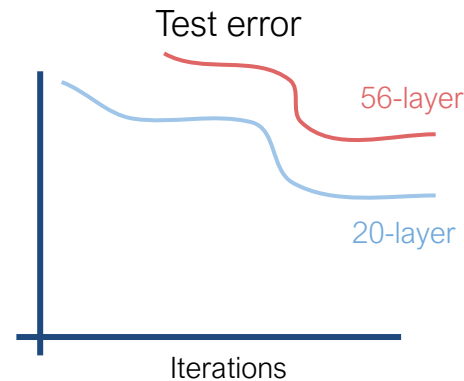
Once we have Batch Normalization, we can train networks with 10+ layers. What happens as we go deeper?

Residual Networks

Once we have Batch Normalization, we can train networks with 10+ layers.
What happens as we go deeper?

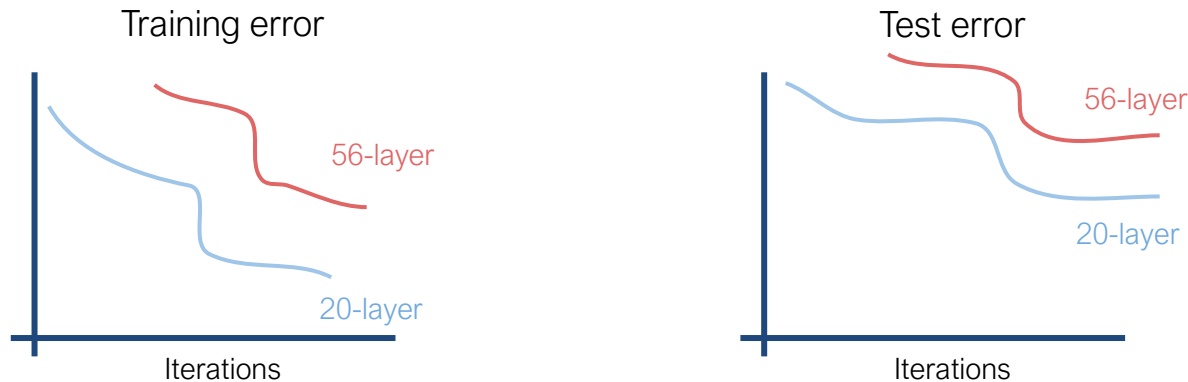
Deeper model does worse than shallow model!

Initial guess: Deep model is **overfitting** since it is much bigger than the other model



Residual Networks

Once we have Batch Normalization, we can train networks with 10+ layers. What happens as we go deeper?



In fact the deep model seems to be **underfitting** since it also performs worse than the shallow model on the training set! It is actually **underfitting**

Residual Networks

A deeper model can emulate a shallower model: copy layers from shallower model, set extra layers to identity

Thus deeper models should do at least as good as shallow models

Hypothesis: This is an optimization problem. Deeper models are harder to optimize, and in particular don't learn identity functions to emulate shallow models

Residual Networks

A deeper model can emulate a shallower model: copy layers from shallower model, set extra layers to identity

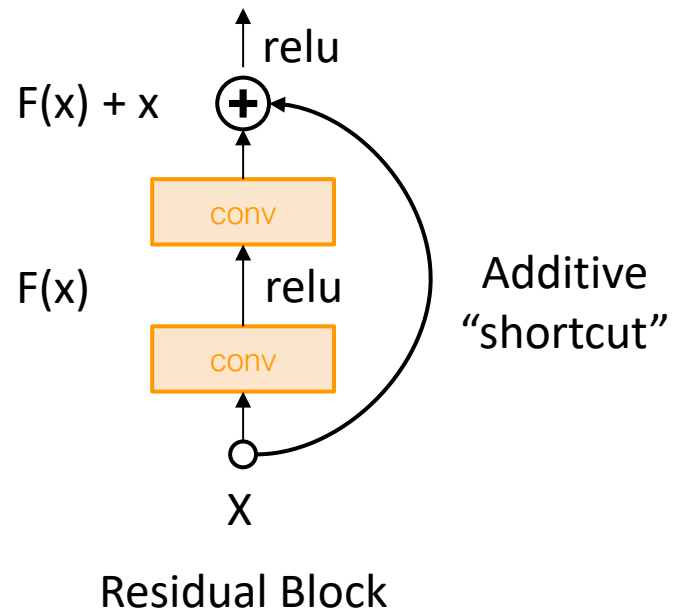
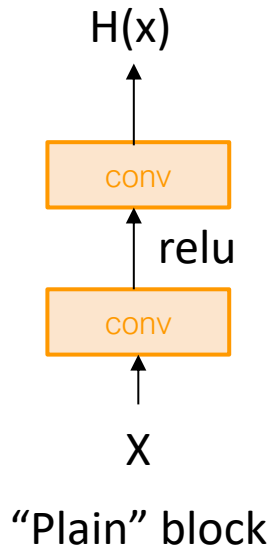
Thus deeper models should do at least as good as shallow models

Hypothesis: This is an optimization problem. Deeper models are harder to optimize, and in particular don't learn identity functions to emulate shallow models

Solution: Change the network so learning identity functions with extra layers is easy!

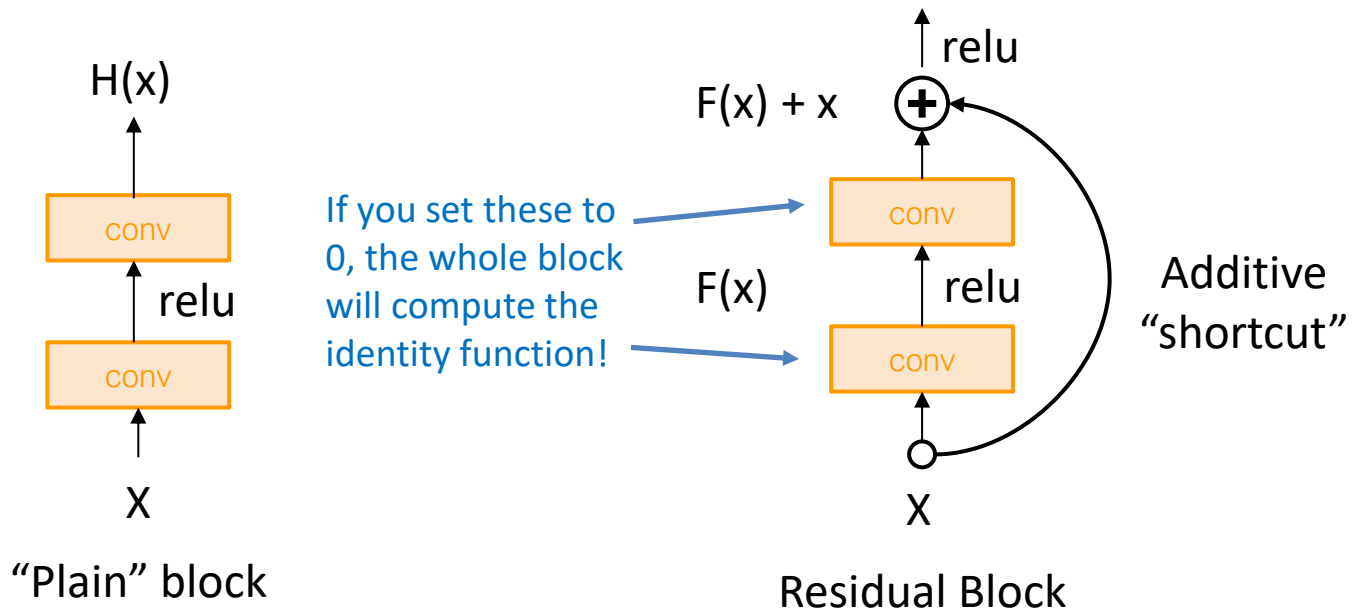
Residual Networks

Solution: Change the network so learning identity functions with extra layers is easy!



Residual Networks

Solution: Change the network so learning identity functions with extra layers is easy!

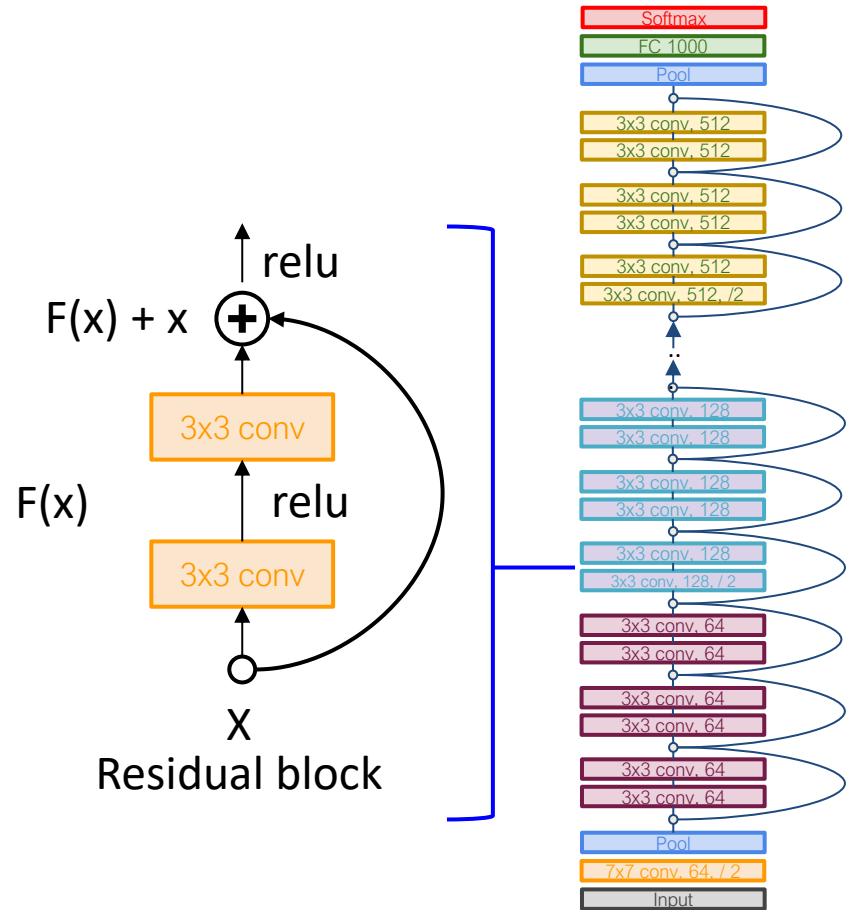


Residual Networks

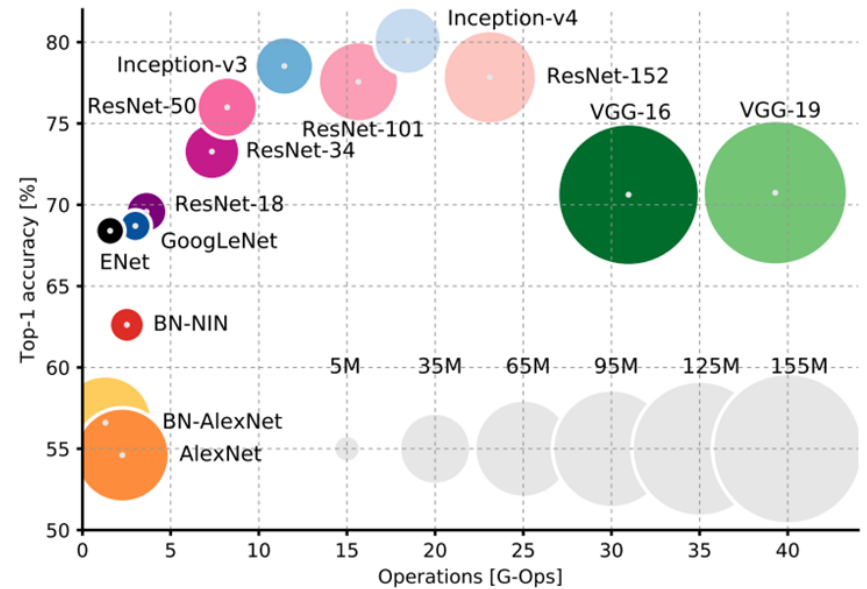
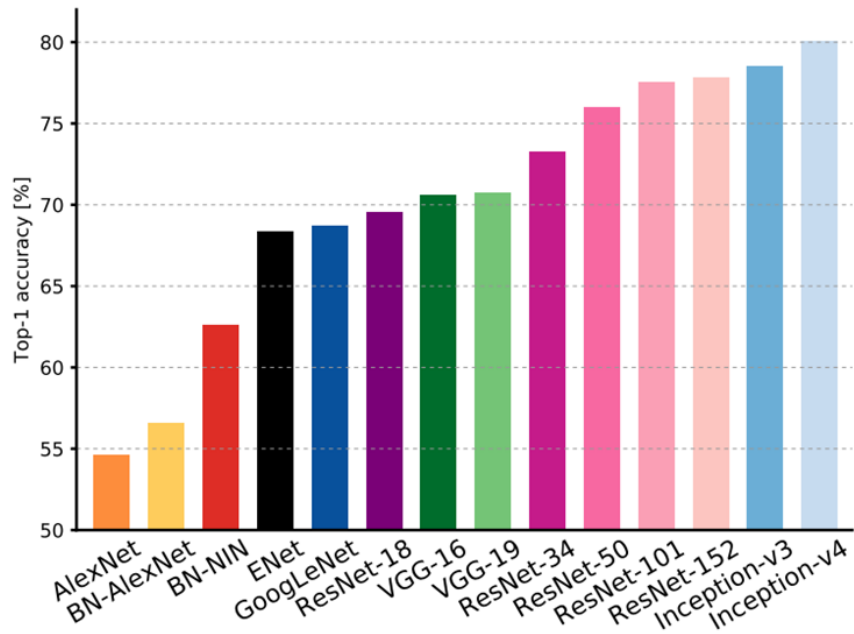
A residual network is a stack of many residual blocks

Regular design, like VGG: each residual block has two 3x3 conv

Network is divided into **stages**:
the first block of each stage halves the resolution (with stride-2 conv) and doubles the number of channels



Comparing Complexity



BN: batch normalized version